

ADR-3 Selección de Tecnologías

Situación

Aceptado

Contexto

El sistema Geojana requiere definir el stack tecnológico de desarrollo para implementar la arquitectura basada en servicios establecida en ADR-2. El sistema comprende dos frontends PWA (aplicación de usuario orientada a móvil, y plataforma de administración e institución orientadas a escritorio) y múltiples servicios que exponen APIs REST consumidas a través de un gateway.

Las restricciones que condicionan esta decisión son:

- No se puede utilizar PHP ni React.
 - El presupuesto de infraestructura debe ser mínimo.
 - El tiempo de desarrollo es limitado.
 - La solución debe soportar funcionamiento offline con sincronización posterior, GPS, cámara y notificaciones push, según lo propuesto en los ADR-1 y ADR-2.
 - Los servicios de dominio deben exponer contratos REST documentados, verificables mediante pruebas de integración.
-

Decisión

Se decide una combinación tecnológica que prioriza la ligereza en el cliente y la eficiencia en el enrutamiento: Vue.js para el frontend, Python para los servicios de dominio, **Node.js para la Capa de API** y Supabase para la persistencia de datos.

Alternativas Evaluadas

Frontend

Alternativa 1 — Vue.js

- **Criterios:** Facilidad de implementación, soporte PWA, integración con Leaflet y renderizado condicional.
- **Ventajas:** Curva de aprendizaje suave y un ecosistema maduro. Su herramienta Vite permite configurar PWAs nativas de forma muy rápida y maneja el estado global de manera predecible, lo cual es crítico para gestionar la caché de reportes offline. Permite crear interfaces de administración limpias y rápidas.
- **Desventajas:** Para las aplicaciones de instituciones o administradores orientadas puramente a escritorio, podría requerir integración con herramientas secundarias (como Electron o Tauri) si la PWA instalable no es suficiente.

Alternativa 2 — Svelte

- **Criterios:** Rendimiento extremo, peso de la aplicación y eficiencia de carga.
- **Ventajas:** A diferencia de otros frameworks, Svelte compila el código a JavaScript puro en lugar de usar un Virtual DOM. Esto genera aplicaciones increíblemente ligeras y rápidas, lo cual es ideal para asegurar la operatividad de la PWA en zonas rurales con conectividad limitada.
- **Desventajas:** Su comunidad es más pequeña, lo que implica que la integración con la librería de mapas (Leaflet) deberá hacerse interactuando directamente con el DOM nativo en lugar de depender de componentes prefabricados.

Backend

Alternativa 1 — Python con FastAPI o Django

- **Criterios:** Ecosistema geográfico, validación de reglas de negocio y documentación.
- **Ventajas:** El ecosistema de Python para manejar datos geográficos (GeoJSON, cálculos de distancia, intersección de polígonos) es inmensamente superior y maduro. FastAPI ofrece una velocidad excepcional y autogenera la documentación de los contratos de API, lo que agiliza la comunicación entre el equipo de frontend y backend. Django (en modo API) proporciona una estructura muy robusta si se requiere validación estricta para el manejo de usuarios o moderación.
- **Desventajas:** Requiere disciplina en la gestión entornos virtuales o Docker para evitar conflictos de dependencias entre servicios.

Alternativa 2 — Node.js con Express o Fastify

- **Criterios:** Unificación del lenguaje, altísimo rendimiento de red y concurrencia.
- **Ventajas:** Si el frontend está en Vue o Svelte (JavaScript/TypeScript), usar Node.js en el backend reduce la fricción cognitiva. Fastify es uno de los frameworks web más rápidos que existen para Node.js, ideal para manejar un alto volumen de peticiones simultáneas.
- **Desventajas:** Aunque existen librerías para manejar datos espaciales en Node.js, no son tan maduras ni robustas como las de Python lo que implica escribir más consultas SQL en texto plano. La generación de documentación (Swagger) y la validación de esquemas requieren instalar y configurar plugins adicionales manualmente.

API Gateway

Alternativa 1 — Node.js con Express o Fastify

Criterios: Velocidad de enrutamiento, concurrencia y manejo de red.

Ventajas: Su naturaleza asíncrona es perfecta para actuar como el punto de entrada único del sistema. Es altamente eficiente para recibir miles de peticiones de las PWAs, verificar los tokens de seguridad y enrutar el tráfico hacia los puertos de los servicios internos correspondientes sin bloquear procesos.

Desventajas: No es ideal para procesamiento intensivo de datos en CPU, aunque el rol de Gateway solo requiere operaciones de red (I/O).

Alternativa 2 — Nginx

- **Criterios:** Rendimiento extremo, estandarización de la industria y uso de recursos.
- **Ventajas:** Puede manejar decenas de miles de conexiones consumiendo pocos mb de RAM, es la herramienta definitiva para proxies inversos. No requiere lógica de negocio, solo la configuración del

archivo base. Es ideal para servir los archivos compilados (HTML/CSS/JS) del PWA de Vue o Svelte. Además, el balanceo de carga y caché vienen integrados de forma nativa.

- **Desventajas:** Configuración estricta, un error de sintaxis puede comprometer el servidor. La seguridad es compleja debido a que Nginx no lee bases de datos ni descripta JWTs fácilmente por sí solo. Para validar un usuario, debes configurar una arquitectura de auth_request de forma manual usando un tercer servicio.

Base de Datos

Alternativa 1 — Supabase

- **Criterios:** Capacidades espaciales nativas, velocidad de desarrollo y gestión de permisos (RLS).
- **Ventajas:** Integra PostGIS para consultas de geolocalización. La gestión de autenticación mantiene el esquema limpio (sin requerir tablas o columnas para gestionar refresh tokens manualmente). Las políticas RLS aseguran el aislamiento de datos entre instituciones regionales. La curva de aprendizaje es mínima al poder replicar la experiencia en el diseño de esquemas y lógica de datos que ya se domina en esta plataforma.
- **Desventajas:** No posee un motor de sincronización offline automático, lo que obliga a implementar Service Workers e IndexedDB manualmente en el frontend para guardar reportes cuando no haya conexión.

Alternativa 2 — PostgreSQL + PostGIS

- **Criterios:** Control absoluto, bajo costo de infraestructura bruta y nula dependencia de proveedores.
- **Ventajas:** Flexibilidad arquitectónica total sin límites comerciales de plataformas de terceros. El costo mensual es fijo y predecible mediante un servidor VPS, independientemente del volumen de usuarios o reportes procesados simultáneamente.
- **Desventajas:** Alta sobrecarga operativa. Requiere desarrollar desde cero toda la infraestructura de autenticación y asumir la responsabilidad completa del mantenimiento, parches de seguridad y respaldos de la base de datos.

Alternativa 3 — Firebase Firestore

- **Criterios:** Sincronización offline en tiempo real y facilidad de implementación móvil/web.
- **Ventajas:** Resuelve nativamente el problema de conectividad en zonas rurales mediante un excelente soporte offline que encola y sincroniza los reportes automáticamente al recuperar la señal. Permite actualizaciones en tiempo real en los paneles institucionales sin configuraciones extra.
- **Desventajas:** Al ser NoSQL, carece de soporte espacial avanzado para calcular distancias, intersecciones o radios de forma eficiente. Exige el uso de herramientas de terceros o procesamiento de coordenadas en memoria que complican y degradan el rendimiento del backend.

Consecuencias

- **Rendimiento en zonas rurales (+):** Descartar frameworks pesados y optar por Vue/Svelte garantiza que la PWA cargue rápidamente incluso con redes inestables.

- **Seguridad y Cohesión (+):** Node.js actúa como escudo de seguridad, mientras que Python maneja la lógica compleja de manera aislada, mejorando la mantenibilidad.
 - **Despliegue (+/-):** Este stack requerirá el uso de contenedores para empaquetar Node.js y Python de manera predecible, lo que añade un ligero esfuerzo inicial en infraestructura.
-

Cumplimiento

- Verificación de que el tamaño del paquete (bundle) inicial del frontend no exceda los umbrales recomendados para PWAs en conexiones 3G/4G.
 - Pruebas de latencia en el API Gateway para asegurar que el enrutamiento y validación de seguridad añada un impacto mínimo de red.
 - Validación de que las coordenadas de Leaflet se almacenen y consulten correctamente utilizando los tipos de datos espaciales de PostGIS.
-

Notas

Autor: @VIVIANA ANDREA CASTRO PINCHEIRA

Día de publicación: 24-05-2026

Última actualización: 24-05-2026